DISTRIBUTED SYSTEMS · ASSIGNMENT 2

گزارش فنی پلتفرم بانکداری توزیع‌شده

مخزن پروژه: https://github.com/SArshiaK/Distributed-Systems/tree/main/2

به‌جای Docker Hub، همهٔ ایمیج‌های پروژه روی GitHub Container Registry منتشر شده‌اند: https://github.com/SArshiaK?tab=packages&repo_name=Distributed-Systems

این گزارش معماری را از یک مسیر سادهٔ درخواست شروع می‌کند و در هر بخش یک لایهٔ واقعی سیستم را اضافه می‌کند. در پایان، تصویر کامل سرویس‌ها دیده می‌شود: Gateway برای مسیریابی، Shard Map برای نگاشت حساب به shard، Replicaها برای تحمل خطا، Coordinator برای تراکنش‌های بین‌shard، Migrator برای مهاجرت زنده، و Healer برای بازیابی خودکار.

فهرست مطالب

1. مقدمه، شبکه و پورت‌ها
2. مرحله ۱ — هسته
3. مرحله ۲ — Sharding
4. مرحله ۳ — Replica و Load Balancing
5. مرحله ۴ — محلی / Cross-Shard
6. مرحله ۵ — 2PC
7. تصمیم ماندگار و Recovery
8. همزمانی و صحت
9. Bonus I — Live Re-Shard
10. Bonus II — Self-Healing
11. آزمایش‌ها و نتایج

۱. مقدمه، هدف و پوشش الزامات

هدف سیستم، سرویس بانکداری توزیع‌شده روی یک میزبان با Docker Compose است: حساب‌ها بین دو shard مستقل پارتیشن می‌شوند، هر shard چند replica برنامه‌ای روی یک دیتابیس مشترک دارد، Gateway مسیر را از روی Shard Map پیدا می‌کند، و انتقال بین‌shard با Coordinator و پروتکل Two-Phase Commit هماهنگ می‌شود. کلاینت فقط با Gateway حرف می‌زند و از جزئیات shard و replica بی‌خبر است.

۱.۰ شبکه و پورت‌ها

همهٔ سرویس‌های تعریف‌شده در `docker-compose.yml` روی **یک شبکهٔ پیش‌فرض Compose** بالا می‌آیند (نام پروژه `2` → شبکهٔ `2_default`). شبکهٔ جداگانه‌ای تعریف نشده است؛ بنابراین کانتینرها با **DNS داخلی روی نام سرویس** همدیگر را پیدا می‌کنند (مثلاً `http://gateway:8080` یا `postgresql://bank:bank@db-a:5432/bank`). ارتباط سرویس‌به‌سرویس از داخل شبکه انجام می‌شود و به پورت میزبان وابسته نیست.

پورت داخل کانتینر همان پورت پیش‌فرض اپ است (بانک‌ها `8000`، Gateway `8080`، و غیره). روی میزبان، برای هر سرویسی که باید از بیرون دیده شود یک پورت یکتا نگاشت شده تا با هم تداخل نکنند و برای UI، تست و دیباگ قابل دسترسی باشند. دیتابیس‌های Postgres به میزبان publish نشده‌اند و فقط داخل همان شبکه روی `5432` در دسترس‌اند.

سرویس	پورت میزبان → داخل	نقش
`bank-a1` … `bank-b2`	`8001–8004 → 8000`	replicaهای shard A/B (ترتیب متوالی روی میزبان)
`bank-c1` / `bank-c2`	`8005–8006 → 8000`	replicaهای shard C (profile bonus)
`gateway`	`8080 → 8080`	ورود کلاینت / آزمایش‌ها
`coordinator`	`8090 → 8090`	2PC
`migrator`	`8095 → 8095`	Bonus I
`healer`	`8096 → 8096`	Bonus II
`console`	`3030 → 3000`	Lab UI
`db-a/b/c/coord`	بدون publish به میزبان	فقط داخل شبکهٔ Compose روی 5432

۱.۱ نگاشت اجزای ASSIGNMENT به پیاده‌سازی

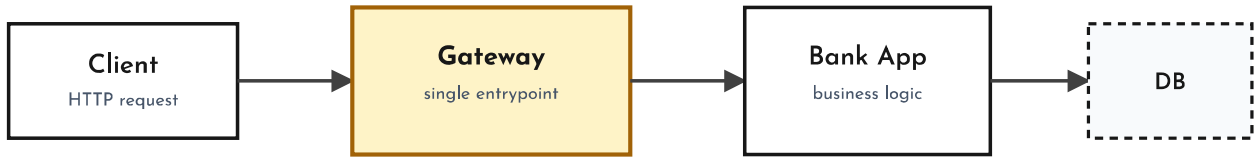
#	الزام ASSIGNMENT	پیاده‌سازی
1	Banking Client	نقش کلاینت بانک در دو شکل پیاده شده است: رابط وب Lab Console روی پورت `3030`؛ برای آزمایش دستی، و اسکریپت‌های پوشهٔ `experiments/` برای سناریوهای خودکار. هر دو فقط به Gateway درخواست می‌فرستند و از وجود shard، replica یا Coordinator بی‌خبرند.
2	Shard-Aware Gateway	سرویس `gateway/` حساب مبدأ و مقصد را از Shard Map می‌خواند، shard مسئول را انتخاب می‌کند، و بین replicaهای سالم همان shard با Round-Robin توزیع بار می‌کند. اگر انتقال بین‌shard باشد، به‌جای ارسال مستقیم به بانک، درخواست را به Coordinator می‌سپارد تا 2PC اجرا شود.
3	Banking Shards A/B	دو shard مستقل با دیتابیس‌های `db-a` و `db-b` وجود دارد. برای هر shard دست‌کم دو replica برنامه‌ای (bank-server) بالا می‌آید که همگی به همان Postgres مشترک وصل‌اند؛ بنابراین موجودی authoritative فقط در دیتابیس shard است، نه در حافظهٔ هر replica.
4	Transaction Coordinator	سرویس `coordinator/` تراکنش‌های cross-shard را با پروتکل Two-Phase Commit هدایت می‌کند. وضعیت ماندگار هر تراکنش (مثل prepared و تصمیم نهایی) در Postgres جداگانهٔ `db-coord` نوشته می‌شود تا بعد از restart بتواند recovery کند و تراکنش‌های نیمه‌کاره را کامل یا abort کند.
5	Shard Map	فایل `config/shard_map.json` نگاشت حساب به shard را نگه می‌دارد: بازهٔ شمارهٔ حساب، شناسهٔ shard، و فهرست آدرس replicaها. فیلد `epoch` نسخهٔ نقشه را مشخص می‌کند تا در زمان مهاجرت زنده (Bonus I) Gateway و Migrator روی یک نسخهٔ سازگار توافق کنند.
6	آزمایش‌های ۱–۷	پوشهٔ `experiments/` سناریوهای اجباری assignment را پوشش می‌دهد. اسکریپت `run_all.py` همه را پشت‌سرهم اجرا می‌کند و هر آزمایش جداگانه هم اسکریپت مخصوص خود را دارد تا بتوان مسیرهای سلامت، قطع replica، و خطاهای 2PC را جداگانه بررسی کرد.
7	Bonus I / II	Bonus I از طریق سرویس `migrator` پیاده شده است و با Docker Compose profile به نام `bonus` اجرا می‌شود. این سرویس حساب‌ها را بین shardها جابه‌جا می‌کند بدون اینکه کل سیستم خاموش شود. Bonus II از طریق سرویس `healer` پیاده شده است؛ این سرویس replicaهای خراب را پیدا می‌کند و با منطق خودمان دوباره راه‌اندازی می‌کند.

۲. مرحله ۱ — هستهٔ ساده

در اولین لایه فقط یک مسیر وجود دارد: کلاینت به Gateway می‌رسد، Gateway به یک سرویس بانکی می‌فرستد، و بانک روی Postgres می‌نویسد. هنوز sharding و 2PC نیست؛ فقط قرارداد «یک ورودی برای همهٔ درخواست‌ها» ثابت می‌شود.

STAGE 01 · CORE PATH

شکل ۱ · هستهٔ درخواست



عملیات پایه: transfer · withdraw · deposit · balance از یک API واحد.

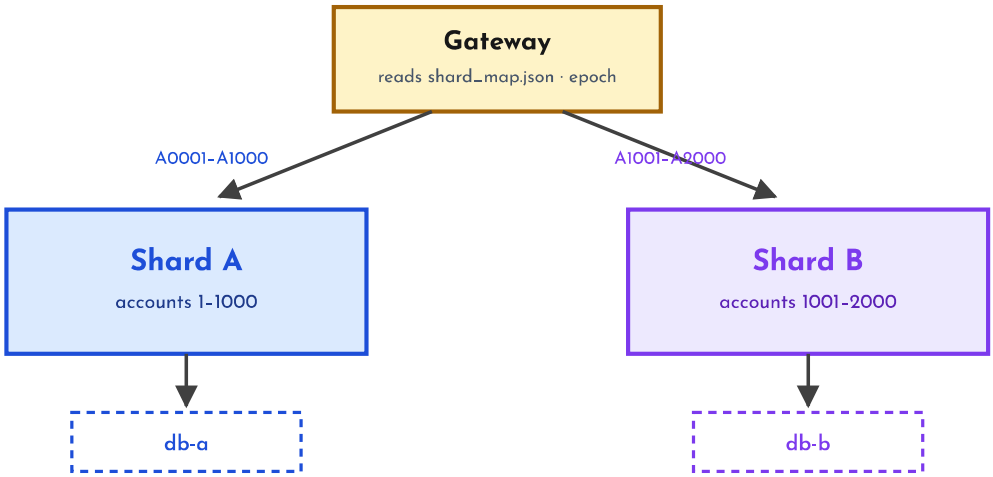
۳. مرحله ۲ — SHARDING و SHARD MAP

فضای حساب‌ها با شناسهٔ Axxxx به بازه‌ها شکسته می‌شود و در `config/shard_map.json` ثبت می‌گردد. Gateway با شمارهٔ حساب، shard مالک را پیدا می‌کند. فیلد epoch نسخهٔ منطقی map است و هنگام مهاجرت زنده افزایش می‌یابد.

- db-a → **Shard A:** ۱–۱۰۰۰
- db-b → **Shard B:** ۱۰۰۱–۲۰۰۰
- **Shard C (بعد از Bonus I):** معمولاً ۱۵۰۱–۲۰۰۰ ← db-c

STAGE 02 · SHARD MAP

شکل ۲ · افزودن SHARDING



Shard Map منبع حقیقت مسیریابی است؛ epoch جلوی استفاده از نقشهٔ کهنه را می‌گیرد.

۴. مرحله ۳ — REPLICAها و LOAD BALANCING

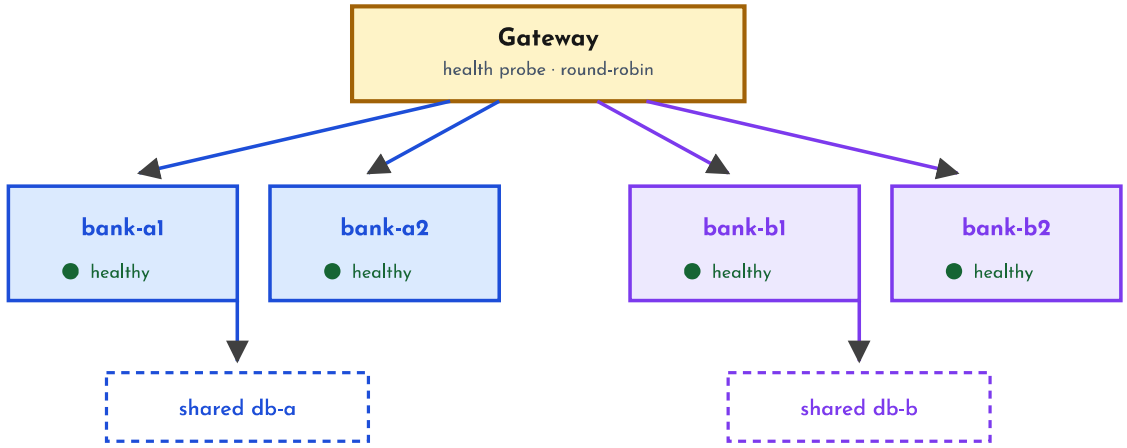
هر shard دست‌کم دو نمونهٔ برنامه دارد که به یک دیتابیس مشترک وصل‌اند (مثلاً bank-a1 و bank-a2 روی db-a). Replicaها موجودی جداگانه نگه نمی‌دارند؛ فقط لایهٔ اجرای منطق کسب‌وکار و اتصال به دیتابیس مالک هستند. Gateway

حدود هر ۲ ثانیه وضعیت سلامت را از endpoint با نام `health` می‌پرسد، نتیجه را در حافظه نگه می‌دارد و فقط بین replicaهای سالم همان shard با **Round Robin** انتخاب می‌کند. اگر حین ارسال درخواست یک replica قطع شود، Gateway آن را ناسالم علامت می‌زند. در نتیجه وقتی یکی از replicaهای یک shard از کار بیفتد، ترافیک همان shard فقط به replica سالم باقی‌مانده می‌رود؛ این رفتار در آزمایش ۴ بررسی می‌شود و همان فرضی است که Bonus II برای جایگزینی خودکار replica روی آن بنا شده است.

پارتیشن‌بندی range-based و قطعی است: عدد داخل `Axxxx` در بازهٔ map می‌افتد. درخواست یک حساب هرگز برای balance جهانی به shard نامربوط پخش نمی‌شود؛ همیشه اول مالک، بعد پخش بار داخل همان مالک.

STAGE 03 · REPLICAS

شکل ۳ · REPLICA و HEALTH-AWARE RR



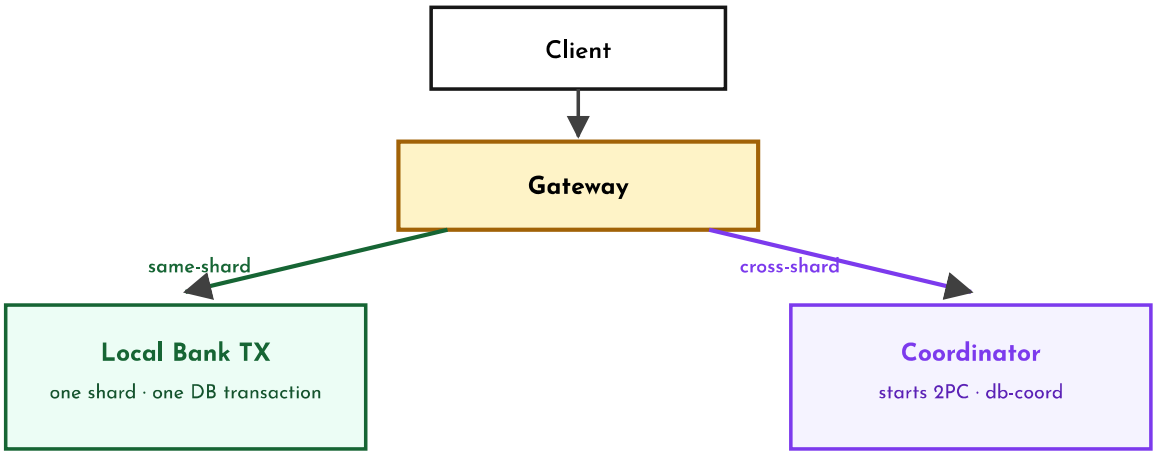
موازنه بار در Gateway است؛ صحت داده به‌خاطر دیتابیس مشترک هر shard حفظ می‌شود.

۵. مرحله ۴ — تراکنش محلی در برابر CROSS-SHARD

اگر مبدأ و مقصد transfer در یک shard باشند، Gateway مستقیم به یک replica همان shard می‌فرستد. اگر دو shard متفاوت باشند، درخواست به Coordinator می‌رود تا 2PC اجرا شود—Gateway دیگر خودش پول را جابه‌جا نمی‌کند.

STAGE 04 · LOCAL VS CROSS

شکل ۴ · دو مسیر اجرا



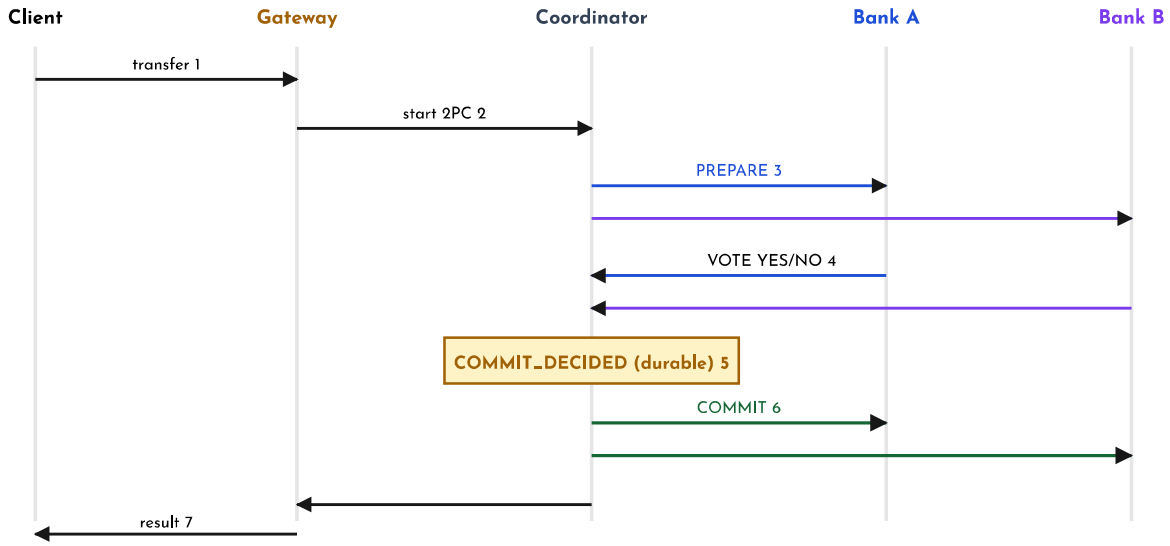
تفکیک مسیر، پیچیدگی توزیع‌شده را فقط جایی می‌آورد که واقعاً لازم است.

۶. مرحله ۵ — پروتکل 2PC و توالی پیام‌ها

Coordinator وضعیت هر تراکنش را در `db-coord` نگه می‌دارد. مسیر موفق به‌ترتیب این است: ابتدا وضعیت `PREPARING` ثبت می‌شود و از هر دو shard رأی گرفته می‌شود؛ اگر همه رأی مثبت بدهند، تصمیم نهایی به‌صورت ماندگار با وضعیت `COMMIT_DECIDED` نوشته می‌شود؛ سپس پیام `COMMIT` به participantها ارسال می‌شود و در پایان تراکنش به `COMPLETED` می‌رسد. اگر حتی یک رأی منفی باشد یا خطایی رخ دهد، به‌جای commit مسیر abort طی می‌شود و تصمیم `ABORT` ثبت و اعمال می‌گردد.

STAGE 05 · TWO-PHASE COMMIT

شکل ۵ · توالی 2PC



تصمیم commit قبل از ارسال COMMIT روی دیسک Coordinator پایدار می‌شود.

#	مسیر	عمل
1	Client → Gateway	درخواست transfer بین‌shard
2	Gateway → Coordinator	شروع 2PC با client_request_id
3-4	Coordinator ⇄ Banks	PREPARE و جمع‌آوری رأی
5	Coordinator / db-coord	ثبت ماندگار COMMIT_DECIDED یا ABORT
6-7	Coordinator → Banks → Client	اعمال نهایی و پاسخ

۷. تصمیم ماندگار، وضعیت‌ها و بازیابی COORDINATOR

هر تراکنش توزیع‌شده یک `transaction_id` یکتا دارد و در `db-coord` با وضعیت‌های استاندارد ذخیره می‌شود: `INITIATED`، `PREPARING`، `PREPARED`، `COMMIT_DECIDED`، `ABORT_DECIDED`، `COMPLETED`. جدول `transaction_participants` رأی و ack هر shard را نگه می‌دارد. API وضعیت با شناسهٔ تراکنش، برای حالت‌های ناتمام `client_status=PENDING/UNKNOWN` برمی‌گرداند و تا وقتی همهٔ participantها COMMIT را تأیید نکرده‌اند، موفقیت نهایی گزارش نمی‌شود.

در شروع سرویس و در حلقهٔ دوره‌ای، Coordinator تراکنش‌های غیرترمینال را از DB می‌خواند و ادامه می‌دهد. شرکت‌کننده‌ها `PREPARE TRANSACTION` / `COMMIT PREPARED` / `ROLLBACK PREPARED` را به‌صورت idempotent پیاده کرده‌اند تا تکرار COMMIT/ABORT بی‌خطر باشد.

۷.۱ نگاشت سناریوهای RECOVERY در ASSIGNMENT

#	سناریو	رفتار سیستم
A	یک shard رأی YES و دیگری رد می‌کند	وضعیت `ABORT_DECIDED` ثبت و برای همهٔ preparedها ABORT فرستاده می‌شود
B	همه prepared شده‌اند و Coordinator می‌میرد (قبل از تصمیم نهایی)	پس از restart، وضعیت‌های `INITIATED/PREPARING/PREPARED` به مسیر abort ایمن می‌روند
C	`COMMIT` ماندگار شده ولی همه ACK نگرفته‌اند	recovery فقط همان COMMIT را تکرار می‌کند؛ هرگز به ABORT برنمی‌گردد (آزمایش ۶)

آزمایش ۶ دقیقاً همین سناریوی C را می‌سازد. با یک فلگ آزمایشگاهی، Coordinator عمداً درست بعد از ثبت ماندگار `COMMIT_DECIDED` و قبل از فرستادن پیام `COMMIT` به بانک‌ها متوقف می‌شود؛ یعنی تصمیم commit در دیتابیس ذخیره شده، ولی هنوز به participantها نرسیده است. وقتی Coordinator دوباره بالا می‌آید، از روی همان تصمیم ذخیره‌شده recovery می‌کند، پیام commit را دوباره می‌فرستد، و تراکنش به `COMPLETED` می‌رسد. موجودی حساب‌ها فقط یک‌بار جابه‌جا می‌شود؛ نه دوبار اعمال می‌شود و نه نیمه‌کاره می‌ماند.

۸. کنترل همزمانی، ACID محلی و صحت

چون چند replica یک shard روی یک Postgres مشترک می‌نویسند، صحت همزمانی در لایهٔ دیتابیس تأمین می‌شود نه با state محلی replica. عملیات نوشتن با `SELECT … FOR UPDATE` روی ردیف حساب قفل می‌گیرند تا lost update رخ ندهد؛ در transfer همون‌shard دو حساب با ترتیب ثابت قفل می‌شوند تا deadlock کمتر شود. کل عملیات داخل یک تراکنش SQL است (atomicity). قید `CHECK (balance >= 0)` و اعتبارسنجی مبلغ مثبت، consistency را در DB و اپ تقویت می‌کند. موفقیت HTTP فقط بعد از commit پایدار Postgres برمی‌گردد (durability).

در مسیر 2PC، وقتی participant فاز PREPARE را اجرا می‌کند، تغییر موجودی را روی دیتابیس اعمال می‌کند اما آن را با `PREPARE TRANSACTION` در حالت نیمه‌کاره نگه می‌دارد؛ یعنی هنوز commit نهایی نشده است. تا وقتی Coordinator تصمیم نگرفته، این تغییر قابل برگشت است: اگر تصمیم commit باشد اثر قطعی می‌شود، و اگر abort باشد کل تغییر rollback می‌شود و موجودی به حالت قبل برمی‌گردد.

در Bonusها دو محافظ اضافه هم هست. اول، هر درخواست کلاینت یک `client_request_id` دارد و پاسخ نهایی در جدول `client_requests` ذخیره می‌شود؛ اگر کلاینت همان درخواست را دوباره بفرستد، سیستم اثر را دوباره اعمال نمی‌کند و همان پاسخ قبلی را برمی‌گرداند. دوم، روی مسیر نوشتن هر shard مقدار `epoch` چک می‌شود؛ بعد از cutover مهاجرت، درخواست‌هایی که هنوز با نقشهٔ قدیمی آمده‌اند رد می‌شوند تا روی مالک قبلی حساب نوشته نشود.

۹. BONUS I — مهاجرت زندهٔ SHARD

در Bonus I، حساب‌های بازهٔ `A1501` تا `A2000` از مالک فعلی یعنی shard B به مالک جدید یعنی shard C منتقل می‌شوند. در طول این کار سیستم خاموش نمی‌شود و ترافیک عادی ادامه دارد. مهم‌تر از همه این است که در هیچ لحظه‌ای دو shard همزمان برای یک حساب ننویسند: یا فقط B می‌نویسد، یا برای مدت کوتاه نوشتن آن بازه فنس می‌شود، یا بعد از cutover فقط C می‌نویسد.

سرویس `migrator` این مهاجرت را مرحله‌به‌مرحله جلو می‌برد. وضعیت هر فاز، چک‌پوینت‌ها، مقدار `epoch` و تصمیم cutover را ماندگار ذخیره می‌کند؛ بنابراین اگر کانتینر migrator وسط کار کرش کند، بعد از بالا آمدن از همان فاز قطع‌شده ادامه می‌دهد و کار را از صفر شروع نمی‌کند.

۹.۱ PROVISION_TARGET

مقصد قبل از هر کپی آماده ولی غیرقابل‌نوشتن می‌شود. Migrator صبر می‌کند replicaهای (`bank-c1/bank-c2`) Shard C healthy شوند، سپس ownership آن‌ها را طوری ست می‌کند که بازهٔ ۱۵۰۱–۲۰۰۰ را «مالک منطقی آینده» بدانند ولی `write_enabled=false` باشد. یعنی C دیتابیس و اپ دارد، اما هنوز نوشتن کلاینت رویش باز نیست. پلتفرم بالا می‌ماند، B همچنان سرویس می‌دهد، و C فقط به‌عنوان هدف خام آماده است.

۹.۲ SNAPSHOT

در این فاز یک کپی اولیه و سازگار از حساب‌های مهاجرت‌شونده از shard B گرفته می‌شود. Migrator از یکی از replicaهای B، موجودی حساب‌های `A1501` تا `A2000` را می‌خواند و هم‌زمان موقعیت فعلی changelog را با نام `change_pos` ذخیره می‌کند. این موقعیت مثل نشانه‌ای است که بعداً مشخص می‌کند catch-up باید از کجا به بعد تغییرات جدید را دنبال کند.

بعد از خواندن، ردیف‌ها تکه‌تکه (chunk) به shard C فرستاده می‌شوند و با upsert ایدمپوتنت نوشته می‌شوند؛ یعنی اگر همان ردیف دوباره اعمال شود، نتیجه خراب یا تکراری نمی‌شود. در طول این فاز ترافیک عادی هنوز روی B ادامه دارد و فقط همین بازه کپی می‌شود؛ حساب‌های خارج از بازه اصلاً درگیر نیستند. برای اینکه بعد از کرش بتوان کار را ادامه داد، تعداد حساب‌ها، جمع موجودی‌ها و همان موقعیت snapshot داخل job مهاجرت ذخیره می‌شود.

CATCHUP ۹.۳

چون هنگام snapshot هنوز روی B نوشتن ادامه داشته، C عقب است. Migrator از changelog مرتب و ماندگار B، تغییرات بعد از `change_pos` را می‌خواند و روی C به‌صورت ایدمپوتنت replay می‌کند. این یک catch-up گرم برای چند ثانیه است تا فاصله کم شود؛ نه کپی یک‌باره و نه صف بی‌کران در حافظه. در این مرحله هنوز فقط B برای بازه می‌نویسد و C دنباله‌رو است، پس دو نویسندهٔ همزمان وجود ندارد.

FENCE_SOURCE ۹.۴

قبل از cutover، نوشتن روی بازهٔ مهاجرت‌شونده در مالک قدیم (B) فنس می‌شود (`fenced_start/end = 1501–2000`). برای همین حساب‌ها B دیگر write جدید قبول نمی‌کند، در حالی که بقیهٔ حساب‌های B (مثل ۱۰۰۱–۱۵۰۰) می‌توانند باز بمانند. این توقف کوتاه فقط برای بستن پنجرهٔ تغییر است تا C به نوک changelog برسد. فنس در مسیر نوشتن خود shard اعمال می‌شود، نه فقط در Gateway.

FINAL_CATCHUP ۹.۵

بعد از فنس تقریباً تغییر جدیدی روی بازه نمی‌آید. Migrator باقی changelog را تا ته خالی می‌کند تا C با B در همان بازه هم‌تراز شود. این مرحله از CATCHUP گرم جداست چون منبع practically آرام شده و می‌توان lag نزدیک صفر را تضمین کرد. هنوز map عمومی به C سوییچ نشده؛ فقط داده و ownership داخلی آمادهٔ cutover است.

WAIT_INFLIGHT_2PC ۹.۶

قبل از عوض‌کردن مالک حساب‌ها، ممکن است هنوز تراکنش‌های cross-shard بازی وجود داشته باشند که به همین حساب‌های مهاجرت‌شونده وصل‌اند؛ مثلاً در وضعیت `PREPARED` یا `COMMIT_DECIDED`. اگر در این لحظه مالک را عوض کنیم، همان تراکنش نیمه‌کاره ممکن است روی shard اشتباه ادامه پیدا کند یا تصمیم قبلی‌اش خراب شود.

برای جلوگیری از این مشکل، migrator قبل از cutover داخل `db-coord` این تراکنش‌های باز را پیدا می‌کند و صبر می‌کند تا هر کدام کامل شوند یا abort شوند. اگر در زمان معقول resolve نشوند، مهاجرت fail می‌شود و cutover انجام نمی‌گیرد. در نتیجه تصمیم ماندگار `COMMIT` دور ریخته نمی‌شود و مالک حساب وسط یک 2PC باز عوض نمی‌گردد.

ACTIVATE_MAP ۹.۷

این فاز لحظهٔ cutover است؛ یعنی مالک رسمی بازه عوض می‌شود. اول تصمیم cutover به‌صورت ماندگار ثبت می‌شود و مقدار `epoch` یکی افزایش پیدا می‌کند تا همه بفهمند نقشهٔ جدید فعال شده است.

بعد از آن، روی shard C بازهٔ ۱۵۰۱ تا ۲۰۰۰ به‌عنوان مالک جدید با `write_enabled=true` و epoch تازه فعال می‌شود. روی shard B مالکیت فقط برای بازهٔ ۱۰۰۱ تا ۱۵۰۰ باقی می‌ماند و فنس موقت نوشتن پاک می‌شود، چون دیگر نیازی به قفل آن بازه نیست. سپس فایل `shard_map.json` به‌روز و منتشر می‌شود و Gateway و Coordinator آن را دوباره می‌خوانند.

از این لحظه درخواست‌های درست برای حساب‌های `A1501` به بعد باید به C بروند. اگر هنوز درخواستی با epoch قدیمی به B برسد، در مسیر نوشتن رد می‌شود. به همین دلیل بعد از cutover هم دو writer همزمان برای یک حساب به‌وجود

نمی‌آید.

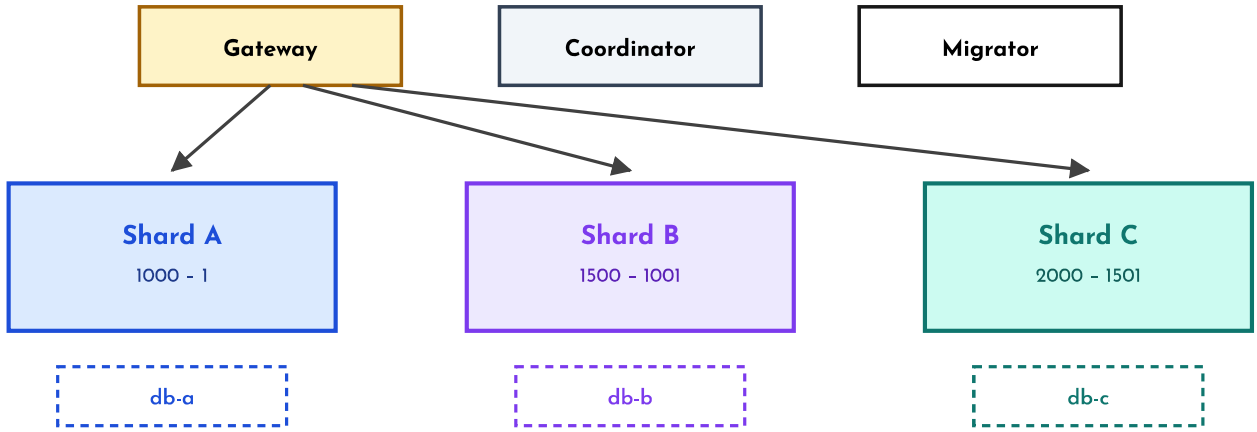
۹.۸ VERIFY و CLEANUP_SOURCE

در VERIFY، migrator صحت cutover را چک می‌کند: آیا C واقعاً حساب‌های بازه را دارد، آیا Gateway شارد C را با همان بازه در map می‌بیند، و آیا مسیریابی با epoch جدید درست کار می‌کند. فقط وقتی این بررسی‌ها قبول شوند، فاز CLEANUP_SOURCE اجرا می‌شود و کپی کهنهٔ همان بازه از B پاک می‌گردد.

اگر VERIFY رد شود، cleanup انجام نمی‌شود تا داده از دست نرود و هنوز نسخهٔ قبلی روی B باشد. وقتی cleanup هم موفق شود، B دیگر نسخهٔ authoritative از `A1501` تا `A2000` ندارد و مهاجرت با وضعیت `DONE` تمام می‌شود.

> جمع‌بندی پروتکل: تا قبل از ACTIVATE فقط B برای بازه می‌نویسد و C فقط دنباله‌رو است. در یک بازهٔ کوتاه فنس، B هم برای آن حساب‌ها نمی‌نویسد تا cutover امن انجام شود. بعد از ACTIVATE فقط C می‌نویسد؛ درخواست‌های کهنه روی B با epoch رد می‌شوند، و در پایان کپی قدیمی از B پاک می‌شود.

شکل ۶ · معماری پس از مهاجرت BONUS I · THREE SHARDS



بعد از epoch، ACTIVATE_MAP جدید مسیر A1501+ را به C می‌فرستد.

۱۰. BONUS II — AUTONOMOUS SELF-HEALING

سرویس Healer وظیفه دارد تعداد replicaهای سالم هر shard را خودش حفظ کند؛ نه با Swarm یا Kubernetes، و نه با restart خودکار Docker. منطق جایگزینی داخل همین سرویس پیاده شده است.

Healer حدود هر ۲ ثانیه وضعیت سلامت هر slot را از endpoint `health` می‌پرسد. اگر یک replica دو بار پشت‌سرهم پاسخ ندهد، نامزد جایگزینی می‌شود. قبل از ساخت کانتینر جدید، نیت جایگزینی (intent) در فایل `healer_state.json` نوشته می‌شود؛ اگر خود Healer وسط کار کرش کند، بعد از بالا آمدن دوباره همان کار را از صفر و تکراری شروع نمی‌کند و کانتینر duplicate ساخته نمی‌شود.

بعد از ثبت intent، کانتینر جایگزین با Docker API ساخته می‌شود، صبر می‌کند تا سالم شود، در Shard Map ثبت می‌گردد و به Gateway و Coordinator اعلام می‌شود تا ترافیک به آن برسد. در پایان، کانتینر خراب حذف می‌شود. در تمام این مدت

replica سالم همان shard به ترافیک پاسخ می‌دهد و سرویس قطع نمی‌ماند.

BONUS II · PROCESS MAP

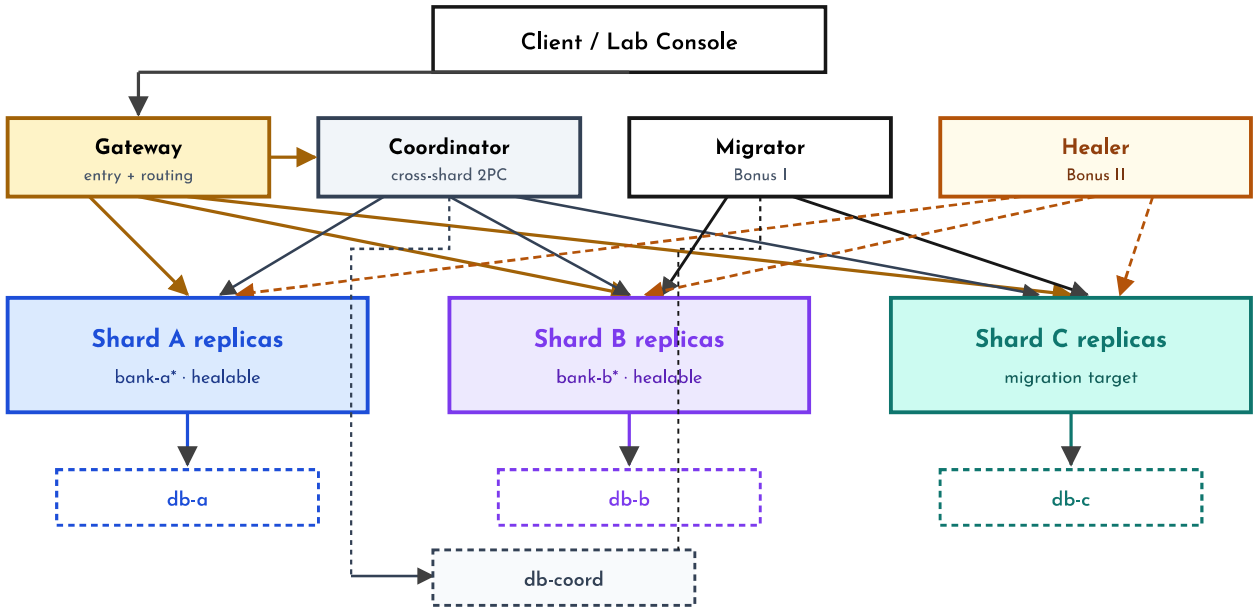
شکل ۷ · جریان SELF-HEALING



شکل ۸ (معماری نهایی) لایهٔ Healer را روی همان قرارداد Shard Map سوار می‌کند.

FULL SYSTEM

شکل ۸ · معماری نهایی



کلاینت فقط به Gateway می‌رود. Gateway به همهٔ shardها route می‌کند و cross-shard را به Coordinator می‌سپارد. Migrator بین B و C کار می‌کند؛ خط‌چین نارنجی Healer برای جایگزینی replica است؛ db-coord جدا از دیتابیس shardهاست.

۱۱. آزمایش‌ها و نتایج

۱۱.۱ آزمایش‌های ۱ تا ۷

#	آزمایش	نتیجهٔ مشاهده‌شده
1	Shard distribution	حساب‌های متعلق به shard A فقط داخل `db-a` هستند و حساب‌های shard B فقط داخل `db-b`. Gateway هر درخواست را به همان shard درست می‌فرستد.
2	Replica load distribution	وقتی چند درخواست پشت‌سرهم برای یک shard می‌آید، ترافیک بین replicaها (مثل A1 و A2) پخش می‌شود و در پاسخ یا لاگ شناسهٔ `replica_id` دیده می‌شود.
3	Concurrent replica ops	واریز و برداشت همزمان از طریق replicaهای مختلف همان shard انجام می‌شود و موجودی نهایی حساب درست و بدون از دست رفتن به‌روزرسانی می‌ماند.
4	Replica interruption	اگر replica دوم (مثلاً A2) متوقف شود، Gateway فقط A1 را انتخاب می‌کند و سرویس همان shard قطع نمی‌ماند. بعد از start دوباره، A2 سالم می‌شود و دوباره در توزیع بار شرکت می‌کند.
5	Local / cross-shard	انتقال داخل یک shard بدون Coordinator انجام می‌شود. انتقال بین‌shard از مسیر 2PC می‌رود و در پایان وضعیت تراکنش در `db-coord` به `COMPLETED` می‌رسد.
6	Interrupted 2PC	Coordinator درست بعد از ثبت `COMMIT_DECIDED` عمداً متوقف می‌شود. بعد از بالا آمدن دوباره، همان تصمیم را ادامه می‌دهد، تراکنش به `COMPLETED` می‌رسد و موجودی فقط یک‌بار جابه‌جا می‌شود.
7	Concurrent cross-shard	چند انتقال بین‌shard همزمان، حتی وقتی حساب‌های مشترک دارند، بدون از دست رفتن پول کامل می‌شوند و مجموع موجودی کل سیستم ثابت می‌ماند.

برای اجرای خودکار این هفت آزمایش، از پوشهٔ `2/experiments` دستور `python run_all.py` را بزنید. سناریوهای Bonus جداگانه با اسکریپت‌های `bonus01_live_reshard.py` و `bonus02_self_heal.py` اجرا می‌شوند.

۱۱.۲ نتایج معماری

- Gateway حساب را به shard درست می‌فرستد؛ هم برای عملیات محلی و هم برای cross-shard.
- اگر یکی از replicaها متوقف شود، peer سالم همان shard به ترافیک ادامه می‌دهد.
- اگر Coordinator بعد از تصمیم ماندگار کرش کند، بعد از restart همان تصمیم را کامل می‌کند.
- در `Bonus I` بازه زنده از B به C منتقل می‌شود و با catch-up، fence و epoch جدید مالک عوض می‌گردد.
- در `Bonus II`، replica خراب خودش جایگزین می‌شود و بدون قطع سرویس در Gateway ثبت می‌گردد.